

Community Forum — Saved Posts

Build a small but complete full-stack slice: a discussion feed plus an end-to-end **bookmark (Saved Posts)** feature. We care less about size and more about how the layers fit together.

WHAT YOU'LL BUILD	TIME BOX	DELIVERABLE
Saved Posts feature A working forum slice, end to end	4–6 focused hours Ship what works; don't gold-plate	Runnable repo or zip Plus a short NOTES.md

This is a **greenfield** exercise — you build everything from scratch, nothing to read first. We want to see how you design and implement a layered feature: data model, API, authorization, client state, UI, internationalization, and tests. The feature is intentionally "boring CRUD." The interesting part is **how** you build it — correctness under edge cases, clean separation of concerns, and good judgment about what matters.

1 The Product

A course platform has a discussion forum. Students write **posts**; other students **like**, **view**, and **comment** on them. We're adding **Saved Posts**: a student can bookmark a post for later and see a list of everything they've saved. You'll build enough forum to make "Saved Posts" meaningful, then the feature itself.

Roles

student

Read posts, save / un-save, and view their own saved list.

moderator

Everything a student can do, plus see any post (across courses) and remove a post.

AUTHENTICATION MAY BE STUBBED

Read a `userId` and `role` from a request header, signed token, or session. Please **don't** build a full identity / login system — but **do** enforce the access rules in §2 as if it were real.

2 Requirements

Core **MUST HAVE**

- 1 **Seed data.** At least 2 courses, a few students, and several posts spread across courses. A student is only "in" the course(s) they're enrolled in.
- 2 **Feed.** A paginated list of posts *for a course the requesting student belongs to*, newest first.
- 3 **Save / un-save.** A student can save and un-save any post **they're allowed to see**. Saving is **idempotent** — saving twice is a no-op, not an error and not a double-count.
- 4 **Hydrated flags.** Every post in the feed and saved list reports, for the current user, `hasSaved: boolean` and `savesCount: number`.
- 5 **Saved list.** An endpoint returns the current student's saved posts, **most-recently-saved first**, paginated.
- 6 **History preserved.** Un-saving must **not** destroy the record. Use a soft delete; re-saving should **reactivate** the existing record rather than create a duplicate.
- 7 **UI.** A feed view with a bookmark toggle on each post, and a separate "Saved" view (including a sensible empty state).
- 8 **i18n.** All user-facing strings come from a message catalog with **at least two locales**. Pluralize the saves count correctly ("1 save" / "12 saves").
- 9 **Tests.** Unit tests for the save / un-save logic, plus at least one API test covering an **authorization boundary** and the **happy path**.

Authorization rules — these are central

401 **Unauthenticated** request to any forum endpoint.

403 **Student** saving / reading a post in a course they're **not** enrolled in.

404 Request for a **post that doesn't exist**.

OWN A student can only read **their own** saved list — never another user's.

Out of scope **STUB OR OMIT**

Real login / identity, payments, email, file uploads, deployment, and an admin UI.

3 Tech Stack

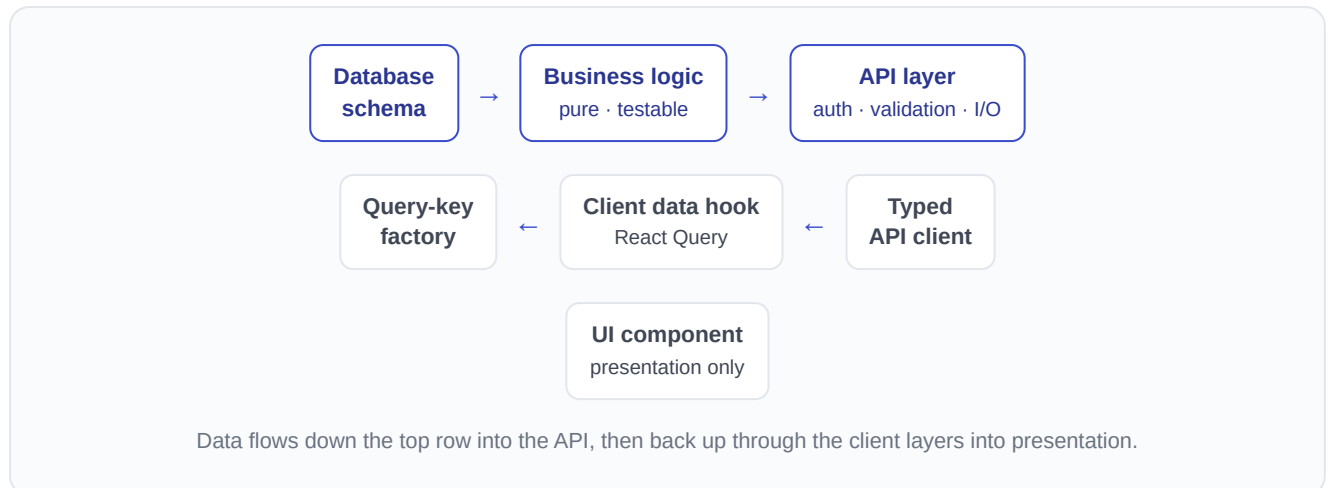
Use the stack below where you can — it lets us evaluate against our production patterns. If you're much stronger elsewhere and want to substitute, that's fine; just note it in [NOTES.md](#). The closer to this list, the better.

LAYER	PREFERRED	ACCEPTABLE SUBSTITUTE
Language	TypeScript (strict mode)	—
Runtime / pkg manager	Bun or Node	—
API	Elysia	Any typed router — Hono, Fastify, Express
Database	PostgreSQL + Drizzle ORM	SQLite + Drizzle, or Prisma
Client state	React Query (TanStack) v5	SWR
UI	React 19 + Next.js (App Router)	Vite + React
Validation	Zod	—
Tests	Vitest + one API / integration test	Jest; Playwright or supertest

A monorepo with a clear [server/](#) and [web/](#) split is welcome but not required — a single Next.js app with route handlers is perfectly fine.

4 How We Think About the Architecture

We care a lot about **layering and separation of concerns**. A clean shape looks roughly like this:



A few things we'll specifically be looking at:

- **Database:** how you model the bookmark relationship and guarantee no duplicate active saves, while still preserving history across un-save / re-save.
- **Business logic:** whether idempotency, count behaviour, and reactivation live in code you can test **without** a database.
- **API:** where auth lives, whether status codes are exact, and how you fetch `hasSaved` / `savesCount` for a list of posts efficiently.
- **Client:** how you manage server state, keep the toggle responsive, and keep the cache consistent after a mutation.
- **UI:** whether presentation stays separate from data fetching, and whether loading / empty states are handled.
- **i18n:** whether strings are externalized and pluralization is correct.

5 How We'll Score It

We grade each area independently and look at the overall shape — a thoughtful, honest submission with clear trade-offs beats a thin layer spread over everything.

AREA	WEIGHT
Data modelling	15%
API & authorization	15%
Business-logic correctness	15%
Tests	15%
Code quality, hygiene & NOTES.md	12%
Client data layer	12%
UI / UX	10%
Internationalization	6%

BASELINE EXPECTATIONS

Strict TypeScript should compile cleanly, and your tests should run from a clean checkout.

6 What to Submit

The code

Runnable from a clean checkout. Fewer manual steps is better. Docker Compose for Postgres is a plus; SQLite to avoid infra is also completely fine.

NOTES.md should cover

- ✓ Key design decisions — schema shape, where auth lives, how you fetch the saved flags efficiently.
- ✓ Trade-offs and anything you deliberately descoped given the time box.
- ✓ What you'd do next with another day.

Setup steps in the README

```
# e.g. bun install
<install>

# create schema and seed data
<migrate + seed>

# start the API
<run server>

# start the UI
<run web>

# run unit + API tests
<test>
```

7 The Follow-Up Conversation

After review, we'll spend about **45 minutes** walking through it together. We'll ask you to trace how a "Save" flows through your system end to end, talk through edge cases (idempotency, concurrency, access control), explain where you chose to put — or not put — certain logic, and sketch how you'd extend the design if requirements changed. There's nothing to prepare beyond being ready to explain and defend your own choices.

8 Ground Rules

Use whatever tools you normally use, including AI assistants. We're hiring for judgment, not memorization — but you should be able to explain every line in the follow-up.

Scope down rather than pad. A clean, well-tested slice beats a sprawling, half-working app.

If you get stuck on setup, stub it, note it in [NOTES.md](#), and keep moving. We're grading the engineering, not your local Postgres install.

Good luck — keep it small, make it correct, and tell us what you traded away.